OpenCV 学习笔记

2026 年 7 月 22 日

> 下载了 ***opencv*** ，***anaconda*** ，了解了 ***python*** 的虚拟环境，下载了 ***vim*** , 并学习了基本用法，查看 ***opencv* 源码**，学习了 ***grep*** 的基本用法, 学习了创建窗口，摧毁窗口，加载图片等基本操作

环境

- 下载 Anaconda，创建虚拟环境
- 下载 OpenCV

编程

学习如何创建窗口

语句	含义	参数	返回值	样例
cv2.namedWindow()	创建一个窗口	第一个参数为窗口的名字，第二个参数为窗口的类型	void	cv2.namedWindow(‘new window’,cv2.WINDOW_NORMAL)
cv2.resizeWindow()	重新设置窗口的大小	窗口名字，宽，高	void	cv2.resizeWindow(‘new window’,640,480)
cv2.imshow(,)	将图像展示到特定的窗口	窗口名字，图像数据	void	cv2.imshow(‘new window’,0)
cv2.waitKey()	等待按键事件发生或者特定的时间	0 就是永久，正数就是时间 (ms)	-1(规定时间到了) 或者按下的按键	key=cv2.waitKey(0)（将按下的按键赋值给 key）
cv2.destroyAllWindows()	关闭所有窗口，释放内存	Null	void	cv2.destroyAllWindows()
cv2.destroyWindow()	关闭这个窗口，释放内存	窗口名字	void	cv2.destroyWindow()

代码

```
import cv2
cv2.namedWindow('new window',cv2.WINDOW_NORMAL)
cv2.resizeWindow('new window',640,480)
cv2.imshow('new window',0)
while True:
    key=cv2.waitKey(0)
    if (key==ord('q')):
        print("q pressed")
        break
print(key)
cv2.destroyAllWindows()
```

学会如何显示图片

语句	含义	参数	返回值	样例
cv2.imread()	读取图片	图片的地址，图片的形式	mat	img=cv2.imread(‘/home/song/图片/壁纸 2.jpeg’,cv2.IMREAD_COLOR)

代码

```
import cv2
cv2.namedWindow('WindowsForImages',cv2.WINDOW_NORMAL)
img=cv2.imread('/home/song/图片/壁纸 2.jpeg',cv2.IMREAD_COLOR)
cv2.imshow('WindowsForImages',img)
while True:
    key=cv2.waitKey(0)
    if key==ord("q"):
        print("P Pressed!")
        break
cv2.destroyWindow('WindowsForImages')
```

终端命令

学会使用 vi 的查找功能

/ + 所需要查找的内容

```
/imread( +ENTER
```

学会使用 grep 查找

grep -rin –include”.*XXX*”“*XXXX*”**.***

- -r：递归搜索
- -i：忽略大小写
- -n: 显示行号
- –include”“是所查找文件名要包含的内容
- *.XXX 是扩展名是 XXX
- “XXXX”是要搜索的内容
- **.** 意味着当前目录

```
grep -rin --include="*.hpp" "imread(" .
```

2026 年 7 月 23 日

学习了保存图片，读取与写入视频文件，获取鼠标事件，创立 track-bar

编程

保存图片

语句	含义	参数	返回值	样例
cv2.imwrite()	保存图片	图片的地址，图片的形式	bool(1: 成功保存，0：失败)	cv2.imwrite(‘/home/song/Code/Python/CV/Pic.pn ,img)

代码

```
import cv2
cv2.namedWindow('WindowsForImages',cv2.WINDOW_NORMAL)
img=cv2.imread('/home/song/图片/壁纸 2.jpeg',cv2.IMREAD_COLOR)
while True:
    cv2.imshow('WindowsForImages',img)
    key=cv2.waitKey(0)
    if key==ord("q"):
        print("P Pressed!")
        break
    elif key==ord('s'):
        cv2.imwrite('/home/song/Code/Python/CV/Pic.png',img)
        print("Picture has been saved!")
```

```
        break
cv2.destroyWindow('WindowsForImages')
```

从摄像头设备或者本地视频文件中捉取视频帧

语句	含义	参数	返回值	样例
cv2.VideoCapture()	打开摄像头获得资源	设备号，摄像头资源的底层接口	返回一个类，这个语句一般赋值给一个变量，之后通过变量来进行操作	capture=cv2.VideoCapture(0,cv2.CA
cv2.VideoCapture()	打开摄像头获得资源	本地视频的地址，摄像头资源的底层接口	返回一个类，这个语句一般赋值给一个变量，之后通过变量来进行操作	capture=cv2.VideoCapture(‘/run/media/song/Data/UESTC/课程/2025.1eagp/Academic English Materials PHASE 1-20250917/CS3/CS3 Talking with your Tutor - Part 1.mp4’,cv2.CAP_ANY)
variable.read()	读取视频帧	null	第一个是是否成功读取帧，第二个是图片（mat格式 u）	ret,frame=capture.read()
variable.release()	释放摄像头/本地视频文件资源	null	void	capture.release()

```
import cv2

# 创建窗口
cv2.namedWindow('video',cv2.WINDOW_NORMAL)

# 获得为了视频打开摄像头，0 是设备（摄像头）编号，CAP_ANY 是为系统自动找到一个合适的 API 接口
capture=cv2.VideoCapture(0,cv2.CAP_ANY)

# 获得为了视频打开摄像头，从视频文件读取视频帧，CAP_ANY 是为系统自动找到一个合适的 API 接口
```

```
capture=cv2.VideoCapture('/run/media/song/Data/UESTC/课程/2025.1eagp/Academic Eng

# 变量
a=0# 保存图片数目
Camera_First=1# 是否第一次获得摄像头资源
while True:
    #capture.read() 两个返回值：第一个是是否成功读取帧，第二个是图片（mat 格式 u）
    ret,frame=capture.read()
    if ret==False:
        if Camera_First==1:
            print("Camera False!")
            Camera_First=0
    elif ret==True:
        if Camera_First==1:
            print("Camera OK!")
            Camera_First=0
    # 显示图片
    cv2.imshow('video',frame)
    # 等待 key
    key=cv2.waitKey(1)
    # 退出
    if key&0xFF==ord('q'):
        print("Quit the window!")
        break
    # 保存
    elif key&0xFF==ord('s'):
        cv2.imwrite('/home/song/Code/Python/CV/Pic/Camera'+str(a)+'.png',frame)
        print("Picture Saved!")
        a=a+1

# 释放资源
capture.release()
cv2.destroyAllWindows()
```

根据视频帧制作视频文件

语句	含义	参数	返回值	样例
variable=cv2.VideoWriter.fourcc()	规定视频编码格式	字符串（编码格式）	类	fourcc=cv2.VideoWriter.fourcc(‘M’,‘J’,‘P’,‘G’)

语句	含义	参数	返回值	样例
variable=cv2.VideoWriter()	将视频文件的信息给类	地址，编码格式，帧数，分辨率，是否是彩色	类	vw=cv2.VideoWriter(‘Pic/MakeVideo.mp4’,fourcc,25,(640,480),isColor=True)
variable.write()	将视频帧写入	mat	void	vw.write(frame)
variable.release()	释放资源	null	void	vw.release()

代码

```
import cv2

# 创建窗口
cv2.namedWindow('video',cv2.WINDOW_NORMAL)

# 写入视频编码格式
fourcc=cv2.VideoWriter.fourcc('M','J','P','G')

# 获得为了视频打开摄像头，0 是设备（摄像头）编号，CAP_ANY 是为系统自动找到一个合适的 API 接口
capture=cv2.VideoCapture(0,cv2.CAP_ANY)

#
vw=cv2.VideoWriter('Pic/MakeVideo.mp4',fourcc,25,(640,480),isColor=True)
# 变量
a=0# 保存图片数目
Camera_First=1# 是否第一次获得摄像头资源
while capture.isOpened():# 判断摄像头是否被打开
    #capture.read() 两个返回值：第一个是是否成功读取帧，第二个是图片（mat 格式 u）
    ret,frame=capture.read()
    if ret==False:
        if Camera_First==1:
            print("Camera False!")
            Camera_First=0
    elif ret==True:
        if Camera_First==1:
            print("Camera OK!")
            Camera_First=0
            # 显示图片
        cv2.imshow('video',frame)
        # 保存到特定地址
        vw.write(frame)
        # 等待 key
        key=cv2.waitKey(1)
        # 退出
        if key&0xFF==ord('q'):
```

```
            print("Quit the window!")
            break
        # 保存
        elif key&0xFF==ord('s'):
            cv2.imwrite('Pic/Camera'+str(a)+'.png',frame)
            print("Picture Saved!")
            a=a+1

# 释放资源
capture.release()
vw.release()
cv2.destroyAllWindows()
```

鼠标事件

语句	含义	参数	返回值	样例
cv2.setMouseCallback() 设置相应的回调函数	设置相应窗口的鼠标回调函数	窗口名字，回调函数，其他参数	void	cv2.setMouseCallback(‘window’,Mouse,param=None)

代码

```
import cv2
import numpy as np
def Mouse(event,x,y,flags,N):
    if event==cv2.EVENT_LBUTTONDOWN:
        print("Left Button Pressed!")
    elif event==cv2.EVENT_RBUTTONDOWN:
        print("Right Button Pressed!")
    elif event==cv2.EVENT_LBUTTONUP:
            print("Left Button Released!")
    elif event==cv2.EVENT_RBUTTONUP:
            print("Right Button Released!")
cv2.namedWindow('window',cv2.WINDOW_GUI_EXPANDED)
cv2.resizeWindow('window',640,480)
img=np.zeros((480,640,3),dtype=np.uint8)
cv2.setMouseCallback('window',Mouse,param=None)
while True:
    cv2.imshow('window',img)
    key=cv2.waitKey(0)
    if key==ord('q'):
        break
cv2.destroyAllWindows()
```

trackbar

语句	含义	参数	返回值	样例
cv2.createTrackbar()	创建滑块	滑块名字，窗口名字，起始值，最大值，回调函数（当滑块位置改变时会调用）	int	cv2.createTrackbar(‘R’,‘window’,0,255,cb)
cv2.getTrackbarPos(‘B’,‘window’)	获得滑块的位置	滑块名称，窗口名称	int(滑块位置)	cv2.getTrackbarPos(‘B’,‘window’)

代码

```
import cv2
import numpy as np
def cb(x):
    pass
# 创建窗口
cv2.namedWindow('window',cv2.WINDOW_NORMAL)
cv2.resizeWindow('window',640,480)

# 创建 trackbar
cv2.createTrackbar('R','window',0,255,cb)
cv2.createTrackbar('G','window',0,255,cb)
cv2.createTrackbar('B','window',0,255,cb)

# 创建 numpy 的元素
img=np.zeros((480,640,3),dtype=np.uint8)

# 显示窗口
while True:
    # 获取 trackbar 的值
    r=cv2.getTrackbarPos('R','window')
    g=cv2.getTrackbarPos('G','window')
    b=cv2.getTrackbarPos('B','window')

    # 赋值
    img[:]=[b,g,r]
    cv2.imshow('window',img)
    key=cv2.waitKey(1)
    if key==ord('q'):
```

```
        break

# 释放资源
cv2.destroyAllWindows()
```

2026 年 7 月 24 日

学习了 Numpy 基础，通道的分割与合并画线

编程

Numpy

语句	含义	参数	返回值	样例
a=np.zeros()	创建一个全 0 的数组	shape，type	类	a=np.zeros((3,2,7),np.uint8)
a=np.ones()	创建一个全 1 的数组	shape，type	类	a=np.ones((3,2,7),np.uint8)
b=np.identity()	创建单位数组	大小，type	类	b=np.identity(8,dtype=np.uint8)
c=np.eye()	创建类单位数组	row，column，para（从第一行的第几个元素开始为 1）	类	c=np.eye(4,6,1)
d=np.full()	创建全为特定值的数组	shape，value,type	类	d=np.full((4,6),12,np.uint8)

代码

```
import numpy as np
a=np.zeros((3,2,7),np.uint8)
b=np.identity(8,dtype=np.uint8)
c=np.eye(4,6,1)
d=np.full((4,6),12,np.uint8)
print(a)
print(b)
print(c)
print(d)
```

Numpy 与图像　将一个 ***numpy*** 数组赋值给 ***img***，后续操作就 **跟普通的一样**

代码

这是一个通过鼠标左右建控制屏幕亮度（颜色）的代码

```
import numpy as np
import cv2
color = 255
img = np.full((480,640),color,np.uint8)
cv2.namedWindow('NumpyWindow',cv2.WINDOW_AUTOSIZE)

def OnMouse(event,x,y,flag,para):
    global color
    # 左键：亮度升高
    if event==cv2.EVENT_LBUTTONDOWN:
        color=(color+50+255)%255
    # 右键：亮度降低
    elif event==cv2.EVENT_RBUTTONDOWN:
        color=(color+255-50)%255
    img = np.full((480,640),color,np.uint8)
    cv2.imshow('NumpyWindow',img)
cv2.setMouseCallback('NumpyWindow',OnMouse,param=None)
while True:
    cv2.imshow("NumpyWindow",img)
    key =cv2.waitKey(0)
    img = np.full((480,640),color,np.uint8)
    if key == ord('q'):
        break
cv2.destroyAllWindows()
```

颜色通道

语句	含义	参数	返回值	样例
cv2.split()	将现有图像分割成三个通道	img	对应的通道的值	b,g,r=cv2.split(img)
cv2.merge((g,r,b))	将不同通道的值和在一起形成一个三通道的数组	（多个通道）	多通道数组	cv2.merge((g,r,b))

代码

```python
import cv2
import numpy

img=cv2.imread('./Pic/Pic.png')
cv2.namedWindow('Pic',cv2.WINDOW_NORMAL)
cv2.resizeWindow('Pic',480,640)
# 分割通道
b,g,r=cv2.split(img)
while True:
    # 合并通道
    cv2.imshow('Pic',cv2.merge((g,r,b)))
    key=cv2.waitKey(0)
    if key==ord('q'):
        break
cv2.destroyAllWindows()
```

画线

语句	含义	参数	返回值	样例
cv2.line()	画线	图片，起始点，终止点，颜色，线的类型	None	cv2.line(img,(320,0),(0,480),(255,0,

代码

```python
import cv2
import numpy as np
cv2.namedWindow('DrawLine',cv2.WINDOW_NORMAL)
img=np.full((480,640,3),255,dtype=np.uint8)
cv2.resizeWindow('DrawLine',640,480)
#img, 起始点，终止点，颜色，线的类型
cv2.line(img,(320,0),(0,480),(255,0,0),cv2.LINE_4)
while True:
    cv2.imshow('DrawLine',img)
    key=cv2.waitKey(0)
    if key==ord('q'):
        break
cv2.destroyAllWindows()
```

2026 年 7 月 27 日

编程

图片上显示文字

语句	含义	参数	返回值	样例
cv2.putText()	在一个窗口的图片上显示文字	窗口名字，文字，文字的位置 (左下角)，字体，字号	void	cv2.putText(img,‘Hello World’,(100,20),cv2.FONT_HERSHEY_PLAIN

```
import cv2
import numpy as np
cv2.namedWindow('text',cv2.WINDOW_NORMAL)
img=np.full((480,640),255,np.uint8)
while True:
    cv2.putText(img,'Hello World',(100,20),cv2.FONT_HERSHEY_PLAIN,3,(0,0,0))
    cv2.imshow('text',img)
    key=cv2.waitKey(0)
    if key==ord('q'):
        break
cv2.destroyAllWindows()
```

图片加法（加减乘除同理）

语句	含义	参数	返回值	样例
add=cv2.add()	将两个图片相加（对应像素的值，超过 255 就保留成 255）	两个图片	add=cv2.add(o1,o2)	

```
import cv2
import numpy as np
cv2.namedWindow('origin1',cv2.WINDOW_NORMAL)
cv2.namedWindow('origin2',cv2.WINDOW_NORMAL)
cv2.namedWindow('add',cv2.WINDOW_NORMAL)
o1=cv2.imread('./Pic/Pic.png')
o2=np.full((847,1275,3),155,np.uint8)
print(o1.shape)
while True:
    add=cv2.add(o1,o2)
```

```
    cv2.imshow('origin1',o1)
    cv2.imshow('origin2',o2)
    cv2.imshow('add',add)
    key=cv2.waitKey(0)
    if key ==ord('q'):
        break
cv2.destroyAllWindows()
```

位运算

语句	含义	参数	返回值	样例
cv2.bitwise_and()	两个图片像素的与运算	img1,img2	mat	cv2.bitwise_and(img1,img2)
cv2.bitwise_or()	两个图片像素的或运算	img1,img2	mat	cv2.bitwise_or(img1,img2)
cv2.bitwise_and()	两个图片像素的异或运算	img1,img2	mat	cv2.bitwise_xor(img1,img2)
cv2.bitwise_not()	两个图片像素的非运算	img1	mat	cv2.bitwise_not(img)

```
import cv2
import numpy as np
cv2.namedWindow('img1',cv2.WINDOW_NORMAL)
cv2.namedWindow('img2',cv2.WINDOW_NORMAL)
img1=np.zeros((480,640),np.uint8)
img2=np.zeros((480,640),np.uint8)
img1[20:100,20:100]=255
img2[70:200,70:200]=255

img_and=cv2.bitwise_and(img1,img2)
img_not=cv2.bitwise_not(img1)
img_or=cv2.bitwise_or(img1,img2)
img_xor=cv2.bitwise_xor(img1,img2)

while True:
    cv2.imshow('img1',img1)
    cv2.imshow('img2',img2)
    cv2.imshow('and',img_and)
    cv2.imshow('not',img_not)
    cv2.imshow('or',img_or)
    cv2.imshow('xor',img_xor)
    key = cv2.waitKey(0)
    if key&0xFF==ord('q'):
        break
cv2.destroyAllWindows()
```

思路

水印

先将对应位置变成黑色，再 ***add*** 上水印

2026 年 7 月 28 日

学习了图像的缩放（resize）和翻转（flip）

编程

缩放图片

语句	含义	参数	返回值	样例
cv2.resize()	将图像缩放到指定大小或按比例缩放	原图, dsize(目标大小), fx(x 轴缩放比例), fy(y 轴缩放比例), interpola-tion(插值方式)	mat	resized=cv2.resize(ori,dsize=None,f

- **dsize** 与 **fx/fy** 二选一: 指定 dsize 则缩放到具体尺寸; 设 dsize=None 并用 fx/fy 则按比例缩放
- **interpolation**（可选）：插值方式，如 cv2.INTER_LINEAR（默认双线性）、cv2.INTER_NEAREST（最近邻）、cv2.INTER_CUBIC（双三次）等

代码

```
import cv2
import numpy as np
ori=cv2.imread('Pic/Pic.png')
new=cv2.resize(ori,dsize=None,fx=0.5,fy=0.5)
cv2.namedWindow('origin',cv2.WINDOW_AUTOSIZE)
cv2.namedWindow('new',cv2.WINDOW_AUTOSIZE)
while True:
    cv2.imshow('origin',ori)
    cv2.imshow('new',new)
```

```
    key = cv2.waitKey(0)&0xff
    if key == ord('q'):
        break
cv2.destroyAllWindows()
```

翻转图片

语句	含义	参数	返回值	样例
cv2.flip()	将图像沿指定轴翻转	原图, flipCode(翻转方式)	mat	flip=cv2.flip(ori,0)

flipCode 参数说明：

值	含义	效果
0	垂直翻转（沿 x 轴）	上下颠倒
1	水平翻转（沿 y 轴）	左右颠倒
-1	同时沿两个轴翻转	上下左右均颠倒

代码

```
import cv2
import numpy as np

ori=cv2.imread('Pic/Pic.png')
cv2.namedWindow('origin',cv2.WINDOW_AUTOSIZE)
cv2.namedWindow('flip',cv2.WINDOW_AUTOSIZE)
flip=cv2.flip(ori,0)
while True:
    cv2.imshow('origin',ori)
    cv2.imshow('flip',flip)
    key=cv2.waitKey(0)&0xff
    if key == ord('q'):
        break
cv2.destroyAllWindows()
```

2026 年 7 月 29 日

学习了图像的仿射变换（warpAffine），实现对图像的平移

编程

仿射变换（平移）

语句	含义	参数	返回值	样例
cv2.warpAffine()	对图像进行仿射变换（平移、旋转、缩放等）	原图, 2×3 变换矩阵 M, 输出尺寸 dsize	mat	new=cv2.warpAffine(img,matrix,(w-150,h-200))

变换矩阵 M 说明：

```
M = [[a, b, tx],
     [c, d, ty]]
```

- 变换公式：`x' = a*x + b*y + tx`，`y' = c*x + d*y + ty`
- **平移矩阵**：`[[1, 0, tx], [0, 1, ty]]`，tx 为水平位移，ty 为垂直位移
- **旋转/缩放**：可用 `cv2.getRotationMatrix2D()` 生成对应的仿射矩阵

注意事项：

- 图像平移后，原图移出的区域会用黑色填充（默认 borderValue=0）
- dsize 指定输出图像的大小，如果比原图小会自动裁剪
- borderMode 可指定边界填充模式（默认 `cv2.BORDER_CONSTANT`），borderValue 设置填充颜色

代码

```python
import cv2
import numpy as np
img=cv2.imread('Pic/Pic.png')
cv2.namedWindow('img',cv2.WINDOW_AUTOSIZE)
cv2.namedWindow('para',cv2.WINDOW_AUTOSIZE)
matrix=np.float32([[1,0,500],[0,1,300]])
h,w,c=img.shape
new=cv2.warpAffine(img,matrix,(w-150,h-200))
while True:
    cv2.imshow('img',img)
    cv2.imshow('para',new)
    key=cv2.waitKey(0)&0xff
    if key ==ord('q'):
        break
cv2.destroyAllWindows()
print(img.shape)
print(new.shape)
```

思路

- `matrix = [[1,0,500],[0,1,300]]`：表示将图像向右平移 500 像素，向下平移 300 像素
- `dsize = (w-150, h-200)`：输出尺寸比原图小，超出部分会被裁剪
- 平移后原图大部分区域移出可视范围，剩余区域被黑色填充

仿射变换（三点确定变换矩阵）

语句	含义	参数	返回值	样例
cv2.getAffineTransform()	根据三组对应点计算 2×3 仿射变换矩阵	src(源图像三点), dst(目标图像三点)	2×3 变换矩阵 M	cv2.getAffineTransform(src,dst)

- 仿射变换需要 **三组不共线的对应点** 来确定变换矩阵（平移 + 旋转 + 缩放 + 剪切）
- 三组点必须为 `np.float32` 类型
- 返回的 2×3 矩阵可直接传入 `cv2.warpAffine()`

示例中的变换含义：

```
src = [[0,0], [0,1], [1,0]]   # 源三角形
dst = [[0,0], [1,0], [0,1]]   # 目标三角形（交换了 x 和 y）
```

该变换将 `(0,1)` 映射到 `(1,0)`，将 `(1,0)` 映射到 `(0,1)`，本质上是 **沿 y=x 对称的反射变换**。

代码

```
import cv2
import numpy as np
import math
img=cv2.imread('Pic/Pic.png')
cv2.namedWindow('win',cv2.WINDOW_NORMAL)
src=np.float32([[0,0],[0,1],[1,0]])
dst=np.float32([[0,0],[1,0],[0,1]])
m=cv2.getAffineTransform(src,dst)
h,w,c=img.shape
new=cv2.warpAffine(img,m,(w,h))
while True:
    cv2.imshow('win',new)
    key=cv2.waitKey(0)&0xff
    if key == ord('q'):
        break
cv2.destroyAllWindows()
```

图像旋转

语句	含义	参数	返回值	样例
cv2.getRotationMatrix2D(center, angle, scale)	获取旋转仿射变换矩阵	旋转中心 center, 角度 angle(度), 缩放比例 scale	2×3 旋转矩阵 M	cv2.getRotationMatrix2D((w/2,h/2),4

- **center**：旋转中心坐标，通常取图像中心 (w/2, h/2)
- **angle**：旋转角度（度），**正值 = 逆时针旋转**
- **scale**：缩放比例，1.0 为不缩放
- 返回的 2×3 矩阵可直接传入 cv2.warpAffine()

旋转矩阵数学形式：

```
M = [[α,  β, (1-α)·cx - β·cy],
     [-β, α,  β·cx + (1-α)·cy]]
```

其中 α = scale·cos(angle)，β = scale·sin(angle)，(cx, cy) 为旋转中心

注意：

- 旋转后图像超出原图尺寸的部分会被裁剪掉
- 如需显示完整旋转图像，需通过 cv2.warpAffine() 的 dsize 参数适当扩大输出尺寸

代码

```python
import cv2
import numpy as np
cv2.namedWindow('window',cv2.WINDOW_AUTOSIZE)
img=cv2.imread('Pic/Pic.png')
h,w,c=img.shape
m=cv2.getRotationMatrix2D((w/2,h/2),45,1.0)
print(m)
rot=cv2.warpAffine(img,m,(w,h))
while True:
    cv2.imshow('window',rot)
    key=cv2.waitKey(0)&0xff
    if key==ord('q'):
        break
cv2.destroyAllWindows()
```

透视变换

语句	含义	参数	返回值	样例
cv2.getPerspectiveTransform()	根据四组对应点计算 3×3 透视变换矩阵	src(源图像四点), dst(目标图像四点)	3×3 透视变换矩阵 M	cv2.getPerspectiveTransform(src,ds
cv2.warpPerspective()	对图像进行透视变换	原图, 3×3 变换矩阵 M, 输出尺寸 dsize	mat	cv2.warpPerspective(img,m,(240,32

与仿射变换的区别：

变换类型	所需点数	变换矩阵	效果
仿射变换	3 组对应点	2×3	保持平行线平行（平移 + 旋转 + 缩放 + 剪切）
透视变换	4 组对应点	3×3	不保持平行线（模拟三维视角、倾斜校正）

- 四点中 **任意三点不能共线**
- 四点通常对应矩形的四个角点，用于 **视角校正** 或 **倾斜矫正**
- cv2.warpPerspective() 的 dsize 指定输出图像的宽度和高度
- 透视变换可以模拟”从斜上方看”的效果，将倾斜的矩形矫正为正视矩形

示例：

```
src = [[88,66], [186,90], [5,208], [150,257]]    # 原图中倾斜矩形的四个角点
dst = [[0,0], [240,0], [0,320], [240,320]]        # 目标矩形的四个角点（正视）
```

该变换将从倾斜视角拍摄的矩形区域校正为 240×320 的正视矩形。

代码

```
import cv2
import numpy as np
cv2.namedWindow('ori',cv2.WINDOW_AUTOSIZE)
cv2.namedWindow('new',cv2.WINDOW_AUTOSIZE)
img=cv2.imread('Pic/pic2.jpeg')
h,w,c=img.shape
src=np.float32([[88,66],[186,90],[5,208],[150,257]])
dst=np.float32([[0,0],[240,0],[0,320],[240,320]])
m=cv2.getPerspectiveTransform(src,dst)
new=cv2.warpPerspective(img,m,(240,320))
while True:
    cv2.imshow('ori',img)
```

```
    cv2.imshow('new',new)
    key=cv2.waitKey(0)&0xff
    if key==ord('q'):
        break
cv2.destroyAllWindows()
```

总结：图像变换家族

变换类型	核心函数	矩阵大小	所需点数	特点
平移	warpAffine + 手动矩阵	2×3	0（直接设 tx,ty）	最简单的位移
旋转/缩放	getRotationMatrix2D + warpAffine	2×3	0（指定中心和角度）	绕中心旋转 + 缩放
仿射（通用）	getAffineTransform + warpAffine	2×3	3 点	保持平行性
透视	getPerspectiveTransform + warpPerspective	3×3	4 点	模拟三维视角

- **仿射变换** 是 **透视变换的特例**：当透视矩阵的 [2][0] 和 [2][1] 元素为 0 时，透视退化为仿射
- **所有变换** 都会导致原图部分区域移出画面（被黑色填充），或者需要进行插值

2026 年 7 月 30 日

学习了图像滤波（平滑/模糊处理）：均值滤波、高斯滤波、中值滤波、双边滤波

概念基础

图像滤波　图像滤波是图像处理中的基本操作，用滤波器（也称为**核** kernel）对图像进行卷积操作，实现**去噪**、**平滑**、**锐化**等效果。

卷积操作原理：用一个小的矩阵（核）在图像上滑动，对每个像素及其邻域像素进行加权平均，得到该位置的新像素值。

滤波的分类

分类	特点	代表
线性滤波	输出像素是输入像素的线性加权组合	均值滤波、高斯滤波

分类	特点	代表
非线性滤波	输出像素与输入像素之间非线性关系	中值滤波、双边滤波

编程

1. 均值滤波（Averaging / Box Filter） 均值滤波是最简单的线性滤波方法，用核区域内所有像素的平均值替代中心像素值。

核心函数：

语句	含义	参数	返回值	样例
cv2.blur()	对图像进行均值滤波	原图, ksize(核大小, 如(5,5))	mat	new=cv2.blur(img,(5,5))
cv2.filter2D()	用自定义卷积核对图像进行卷积	原图, ddepth(输出深度, -1同原图), kernel(卷积核)	mat	new=cv2.filter2D(img,-1,kernel)

核的构造：

```
kernel = np.ones((5,5), dtype=np.float32) / 25  # 5x5 均值核，所有权重均为 1/25
```

- `cv2.blur(img, (5,5))` 等价于用 5×5 的均值核对图像进行卷积
- **ksize** 必须是正奇数，如 (3,3)、(5,5)、(7,7) 等
- **核越大**，平滑效果越强，但图像越模糊，细节损失越多

代码

```
import cv2
import numpy as np

cv2.namedWindow('ori',cv2.WINDOW_AUTOSIZE)
cv2.namedWindow('new',cv2.WINDOW_AUTOSIZE)

img=cv2.imread('Pic/Pic.png')

kernel=np.ones((5,5),dtype=np.float32)/25

#new=cv2.blur(img,(5,5))            # 方式一：直接调用
new=cv2.filter2D(img,-1,kernel)    # 方式二：自定义核卷积
```

```
while True:
    cv2.imshow('ori',img)
    cv2.imshow('new',new)
    key=cv2.waitKey(0)&0xff
    if key==ord('q'):
        break
cv2.destroyAllWindows()
```

---

2. 高斯滤波（Gaussian Blur） 高斯滤波使用符合**高斯分布**的权重核进行卷积。离中心越近的像素权重越大，离中心越远的像素权重越小，因此比均值滤波更”自然”，能更好地保留边缘信息。

核心函数：

语句	含义	参数	返回值	样例
cv2.GaussianBlur()	对图像进行高斯滤波	原图, ksize(核大小), sigmaX(X方向标准差), sigmaY(Y方向标准差)	mat	cv2.GaussianBlur(img,(101,101),sig

参数说明：

- **ksize**：核大小，必须为正奇数。相比均值滤波，高斯滤波常用更大的核（如 101×101）来获得强平滑效果
- **sigmaX / sigmaY**：X/Y 方向的高斯标准差，控制权重的分布范围
  - sigma 越大，权重分布越分散，平滑效果越强
  - `sigmaX=sigmaY=0` 时，OpenCV 会根据 ksize 自动计算 sigma
  - 如果只指定 sigmaX，则 sigmaY 默认与 sigmaX 相同

高斯核数学形式（一维）：

$G(x) = (1 / \sqrt{(2\pi \cdot \sigma^2)}) \cdot \exp(-x^2 / (2\sigma^2))$

二维高斯核是两个一维高斯核的外积。

不同参数的效果对比：

ksize 小	ksize 大	sigma 小	sigma 大
平滑弱，细节保留多	平滑强，细节模糊多	仅中心区域权重高	权重分布广，全局平滑

代码

```
import cv2
import numpy as np

cv2.namedWindow('ori',cv2.WINDOW_AUTOSIZE)
cv2.namedWindow('new',cv2.WINDOW_AUTOSIZE)

img=cv2.imread('Pic/Pic.png')
new=cv2.GaussianBlur(img,(101,101),sigmaX=10.0,sigmaY=10.0)

while True:
    cv2.imshow('ori',img)
    cv2.imshow('new',new)
    key=cv2.waitKey(0)&0xff
    if key==ord('q'):
        break

cv2.destroyAllWindows()
```

---

3. 中值滤波（Median Blur） 中值滤波是**非线性滤波**，用核区域内所有像素的**中位数**替代中心像素值。对**椒盐噪声**（salt-and-pepper noise）有非常好的去除效果。

核心函数：

语句	含义	参数	返回值	样例
cv2.medianBlur()	对图像进行中值滤波	原图, ksize(核大小，必须是奇数整数)	mat	cv2.medianBlur(img,5)

参数说明：

- **ksize**：核大小，**必须为正奇数整数**（如 3、5、7），核形状为 ksize×ksize 的正方形
  - 注意：此处传入的是整数（如 5），不是元组（如 (5,5)）
- ksize 越大，去噪效果越强，但图像越模糊

特点：

- **去椒盐噪声效果极佳**：因为噪声像素通常是极端值（极亮或极暗），中位数不受极端值影响
- **能较好地保留边缘**：不像均值滤波那样模糊边缘
- 处理高斯噪声效果不如高斯滤波

- □ 对大尺寸核计算量较大（需要对每个邻域排序）

代码

```
import cv2
import numpy as np

cv2.namedWindow('ori',cv2.WINDOW_AUTOSIZE)
cv2.namedWindow('new',cv2.WINDOW_AUTOSIZE)

img=cv2.imread('Pic/Pic.png')
new=cv2.medianBlur(img,5)

while True:
    cv2.imshow('ori',img)
    cv2.imshow('new',new)
    key=cv2.waitKey(0)&0xff
    if key==ord('q'):
        break

cv2.destroyAllWindows()
```

---

4. 双边滤波（Bilateral Filter） 双边滤波是**非线性、保边去噪**的滤波方法。它同时考虑**空间邻近度**和**像素值相似度**，因此能在平滑噪声的同时保留边缘。

核心函数：

语句	含义	参数	返回值	样例
cv2.bilateralFilter()	对图像进行双边滤波	原图, d(邻域直径), sigmaColor(颜色空间标准差), sigmaSpace(空间坐标标准差)	mat	cv2.bilateralFilter(img,10,20,30)

参数说明：

参数	含义	越大则
d	滤波时考虑的邻域直径（像素距离）。若 ≤0，则由 sigmaSpace 自动计算	考虑范围越广
sigmaColor	颜色空间标准差。控制像素值差异对权重的影响	颜色差异大的像素也会参与平滑 → 颜色边缘被模糊
sigmaSpace	空间坐标标准差。控制空间距离对权重的影响	远处的像素也会参与平滑 → 更多模糊

工作原理（双重权重）：

双边滤波的权重 = 空间权重 × 颜色权重

$$\text{权重} = \underbrace{\exp(-||p-q||^2/(2\sigma_{space}^2))}_{\text{空间高斯权重}} \times \underbrace{\exp(-|I(p)-I(q)|^2/(2\sigma_{color}^2))}_{\text{颜色高斯权重}}$$

- **空间权重**：与高斯滤波相同，距离中心越近权重越大
- **颜色权重**：像素值与中心像素值越接近权重越大；差异大的像素（如边缘两侧）权重很小
- 因此：**平坦区域** → 空间 + 颜色权重都高 → 平滑效果好；**边缘区域** → 颜色权重低 → 边缘得以保留

代码

```
import cv2
import numpy as np

cv2.namedWindow('ori',cv2.WINDOW_AUTOSIZE)
cv2.namedWindow('new',cv2.WINDOW_AUTOSIZE)

img=cv2.imread('Pic/Pic.png')
new=cv2.bilateralFilter(img,10,20,30)

while True:
    cv2.imshow('ori',img)
    cv2.imshow('new',new)
    key=cv2.waitKey(0)&0xff
    if key==ord('q'):
        break

cv2.destroyAllWindows()
```

总结：四种滤波方法对比

滤波方法	类型	核心函数	核形状	去噪效果	保边能力	适用场景
均值滤波	线性	cv2.blur()	矩形，等权重	★★☆☆☆	★☆☆☆☆	简单平滑，快速去噪
高斯滤波	线性	cv2.GaussianBlur()	矩形，高斯权重	★★★☆☆	★★☆☆☆	通用平滑，高斯噪声
中值滤波	非线性	cv2.medianBlur()	矩形，取中位数	★★★★☆(椒盐)	★★★☆	**椒盐噪声**首选
双边滤波	非线性	cv2.bilateralFilter()	双重权重 (空间 + 颜色)	★★★★☆	★★★★★	**保边去噪**，人像美化

思路

线性滤波 vs 非线性滤波

- **线性滤波**（均值、高斯）：可以用 `cv2.filter2D()` 自定义任意卷积核实现，输出是输入的线性组合
- **非线性滤波**（中值、双边）：不能用简单卷积表示，输出与输入之间是**非线性的**关系

滤波参数的直观理解

- **核大小**：决定”用多大范围的邻居来估计当前像素”。越大 → 越模糊但计算越慢
- **sigma（高斯/双边）**：决定”多远的邻居/多大的颜色差异仍然参与计算”。越大 → 平滑越强

边缘保留的重要性

- 均值滤波和高斯滤波在去噪的同时也会模糊边缘，导致图像**细节丢失**
- 中值滤波和双边滤波在去噪时能较好地保留边缘，适合需要保持图像清晰度的场景
- 双边滤波在**人像美化**、**美颜**中广泛使用——去噪的同时保持皮肤边缘清晰

何时使用哪种滤波?

```
图像包含椒盐噪声 → 中值滤波 (medianBlur)
图像包含高斯噪声 → 高斯滤波 (GaussianBlur)
需要快速简单的平滑 → 均值滤波 (blur)
```

需要保边去噪/美颜效果 → 双边滤波 (bilateralFilter)
需要自定义卷积操作 → filter2D (通用卷积)

2026 年 7 月 31 日

学习了图像的边缘检测：Sobel 算子、Laplacian 算子、Canny 算法，这是图像分割与特征提取的基础

概念基础

什么是图像边缘　**边缘**是图像中灰度值发生剧烈变化的像素集合，通常对应物体的**轮廓**、**边界**，是图像最基本的特征之一。边缘检测的目的就是把”变化剧烈的地方”找出来。

为什么边缘重要：

- 边缘包含了图像的大部分结构信息（轮廓、形状）
- 去掉边缘之外的内容，人眼依然能认出图像的大致内容
- 边缘检测是**目标识别**、**图像分割**、**特征提取**的前置步骤

数学基础：导数与边缘　边缘在数学上对应**灰度函数的变化率**：

概念	定义	与边缘的关系
一阶导数（梯度）	灰度沿某方向的变化率	边缘处梯度幅值大，平坦处接近 0
二阶导数（拉普拉斯）	一阶导数的变化率	边缘处二阶导数过零点（正 ↔ 负）

- **一阶方法**：找到梯度幅值 $|G| = \sqrt{(Gx^2 + Gy^2)}$ 较大的像素 → Sobel
- **二阶方法**：找到二阶导数为 0（过零点）的像素 → Laplacian
- **综合方法**：滤波 + 梯度 + 非极大值抑制 + 双阈值，多阶段优化 → Canny

边缘检测方法分类

类别	原理	代表	特点
一阶微分	梯度幅值取极大值	Sobel / Scharr	实现简单，速度快，边缘较粗
二阶微分	二阶导数过零点	Laplacian / LoG	各向同性，对噪声敏感
多阶段综合	滤波 + 梯度 +NMS+ 双阈值	Canny	边缘细且连续，效果最好

编程

1. Sobel 算子　Sobel 算子是最常用的**一阶微分**边缘检测算子，核心思想：**对图像求导（卷积），导数大的地方就是边缘**。

语句	含义	参数	返回值	样例
cv2.Sobel()	对图像进行 Sobel 求导，计算梯度	原图, ddepth(输出深度), dx(x 方向求导阶数), dy(y 方向求导阶数), ksize(核大小)	mat	cv2.Sobel(img,-1,1,0,ksize=-1)

参数说明：

- **ddepth**：输出图像的深度
  - -1：与输入图像相同（输入是 uint8 则输出也是 uint8）
  - cv2.CV_64F / cv2.CV_32F：浮点深度，**可以表示负数**（求导推荐）
- **dx / dy**：求导的阶数组合
  - dx=1，dy=0：对 x 方向求一阶导 → 检测**垂直边缘**
  - dx=0，dy=1：对 y 方向求一阶导 → 检测**水平边缘**
  - dx 和 dy 不能同时为 0
- **ksize**：核大小，必须为正奇数（1、3、5、7）
  - ksize=-1：使用 **Scharr 算子**（3×3），权重系数更大，对梯度的响应更强、更精确

Sobel 3×3 卷积核：

```
Gx（x方向求导，检测垂直边缘）      Gy（y方向求导，检测水平边缘）
[-1  0  1]                          [-1 -2 -1]
[-2  0  2]                          [ 0  0  0]
[-1  0  1]                          [ 1  2  1]
```

- Gx 中间列为 0：只对左右邻域做差，响应左右灰度突变（垂直边缘）
- 权重 2 是给中心像素加权，自带一点**平滑噪声**的作用，这是 Sobel 比简单差分核（如 [1,0,-1]）更稳定的原因

Scharr 3×3 卷积核（ksize=-1）：

```
Gx                                  Gy
[-3   0  3]                         [-3 -10 -3]
[-10  0 10]                         [ 0   0   0]
[-3   0  3]                         [ 3  10   3]
```

梯度幅值计算：

```
|G| = √(Gx² + Gy²)        精确计算
|G| ≈ |Gx| + |Gy|      近似计算（代码中用 cv2.add(x, y) 实现）
```

代码

```
import cv2
import numpy as np

cv2.namedWindow('ori',cv2.WINDOW_AUTOSIZE)
cv2.namedWindow('new',cv2.WINDOW_AUTOSIZE)

img=cv2.imread('Pic/Pic.png')#uint 8
#dx=1,dy=0: 求 x 方向导数; ksize=-1: 使用 Scharr 算子
x=Sob=cv2.Sobel(img,-1,1,0,ksize=-1)
y=Sob=cv2.Sobel(img,-1,0,1,ksize=-1)
# 用 add 把两个方向的梯度相加（超过 255 截断为 255）
dst=cv2.add(x,y)

while True:
    cv2.imshow('ori',img)
    cv2.imshow('new',dst)
    key=cv2.waitKey(0)&0xff
    if key==ord('q'):
        break

cv2.destroyAllWindows()
```

这段代码的问题（为什么效果”只有一半”）：

- ddepth=-1 意味着输出还是 **uint8**，而梯度值有正有负（由暗到亮是正，由亮到暗是负），uint8 存不下负数，负值全被**截断成 0**
- 结果只能看到”由暗到亮”的边缘（正梯度），“由亮到暗”的边缘丢失了
- cv2.add(x, y) 只是把两个方向的截断结果相加，并没有还原被截断的信息

改进写法（保留正负梯度）：

```
gx=cv2.Sobel(img,cv2.CV_64F,1,0,ksize=3)       # 用 CV_64F 保留负数
gy=cv2.Sobel(img,cv2.CV_64F,0,1,ksize=3)
mag=cv2.addWeighted(cv2.convertScaleAbs(gx),0.5,cv2.convertScaleAbs(gy),0.5,0)
# convertScaleAbs: 取绝对值并转回 uint8，正负边缘都能显示
# addWeighted: 按权重叠加两个方向（各占一半），比 add 更灵活
```

- cv2.convertScaleAbs(): 对 CV_64F 取绝对值，再转回 uint8
- cv2.addWeighted(): 按权重把 x、y 两个方向的梯度叠加
- 顺带一提：x=Sob=cv2.Sobel(...) 是**链式赋值**，x 和 Sob 两个名字指向同一份数据，一般只用一个变量名即可

---

2. Laplacian 算子 Laplacian 算子是一种**二阶微分**边缘检测算子，直接对图像求**二阶导数**。边缘处的二阶导数**过零点**（由正变负或由负变正）。

语句	含义	参数	返回值	样例
cv2.Laplacian()	对图像进行拉普拉斯二阶求导	原图, ddepth(输出深度), ksize(核大小)	mat	cv2.Laplacian(img,-1,ksize=3)

参数说明：

- **ddepth**：输出深度，同上，-1 表示与输入相同（同样有负数截断问题）
- **ksize**：核大小，必须为正奇数（1、3、5、7）；**ksize=1 时使用 3×3 核**

3×3 拉普拉斯核：

```
[ 0  1  0]
[ 1 -4  1]
[ 0  1  0]
```

特点：

- ✅ **各向同性（旋转不变）**：核关于中心对称，不需要像 Sobel 那样分别求 x、y 方向，一次卷积就能检测所有方向的边缘
- ✅ 实现简单，速度与 Sobel 相当
- ❌ **对噪声非常敏感**：噪声点的灰度突变同样会被二阶导放大，所以实际使用前一般先做高斯滤波
- ❌ 边缘两侧会产生一正一负的”双响应”，不如 Canny 干净

高斯-拉普拉斯（LoG）：先 `cv2.GaussianBlur` 去噪，再 `cv2.Laplacian` 求导，两者组合就是 LoG，可以大幅减轻噪声影响。

代码

```python
import cv2
import numpy as np

cv2.namedWindow('ori',cv2.WINDOW_AUTOSIZE)
cv2.namedWindow('new',cv2.WINDOW_AUTOSIZE)

img=cv2.imread('Pic/Camera0.png')
# 直接对彩色图求拉普拉斯：BGR 三个通道分别卷积再合成显示
lap=cv2.Laplacian(img,-1,ksize=3)
cv2.imwrite('Pic/pic3.png',lap)    # 把结果保存到文件
while True:
    cv2.imshow('ori',img)
    cv2.imshow('new',lap)
    key=cv2.waitKey(0)&0xff
```

```
    if key==ord('q'):
        break

cv2.destroyAllWindows()
```

注意：

- 这里直接对**彩色图**求拉普拉斯：OpenCV 会分别对 B、G、R 三个通道做卷积再合成，效果与逐通道处理相同
- `cv2.imwrite` 是之前学过的保存函数，这里把处理结果保存成 `Pic/pic3.png`
- 同样有 `ddepth=-1` 的截断问题,改进方法与 Sobel 相同:`cv2.Laplacian(img,cv2.CV_64F,ksize` 后接 `cv2.convertScaleAbs()`

---

3. Canny 边缘检测 Canny 算法是目前**最经典、最常用**的边缘检测算法，由 John Canny 于 1986 年提出。它不是单一的算子，而是**多阶段处理流程**，目标是最小化漏检率和误检率，输出**单像素宽、连续**的边缘。

语句	含义	参数	返回值	样例
cv2.Canny()	执行 Canny 边缘检测	image(单通道 8 位图), threshold1(低阈值), threshold2(高阈值)	mat	cv2.Canny(gray,90,150)

参数说明：

- **image**：输入图像，**必须是 8 位单通道图（灰度图）**，所以代码里先 `cvtColor` 转灰度
- **threshold1**：**低阈值**（代码里是 90），用于判定”弱边缘”
- **threshold2**：**高阈值**（代码里是 150），用于判定”强边缘”
- 建议 threshold1 与 threshold2 的比值约 1:2 ~ 1:3（90:150 ≈ 1:1.7）

Canny 的五步流程：

原始图像
↓ □ 高斯滤波去噪（消除噪声对梯度的干扰）
↓ □ 计算梯度幅值和方向（内部用 Sobel，方向 = atan2(Gy, Gx)）
↓ □ 非极大值抑制 NMS（沿梯度方向只保留幅值最大的像素 → 边缘细化成单像素）
↓ □ 双阈值检测（>高阈值→强边缘；<低阈值→丢弃；之间→弱边缘）
↓ □ 滞后连接（弱边缘若与强边缘连通则保留，否则丢弃）
输出边缘图

双阈值（滞后阈值）逻辑：

梯度幅值 > 150 → 强边缘，一定保留
90 < 梯度幅值 < 150 → 弱边缘，只有与强边缘相连才保留
梯度幅值 < 90 → 当作噪声，丢弃

- 高阈值太大 → 边缘断断续续（很多真实边缘被当成噪声丢掉）
- 低阈值太小 → 噪声多、边缘粗杂

Canny 为什么效果好：

- 先高斯滤波 → 抗噪
- 非极大值抑制 → 边缘**细**（单像素宽），定位准
- 双阈值 + 滞后连接 → 边缘**连续**，能抑制孤立的假边缘

代码

```
import cv2
import numpy as np

cv2.namedWindow('ori',cv2.WINDOW_AUTOSIZE)
cv2.namedWindow('new',cv2.WINDOW_AUTOSIZE)

img=cv2.imread('Pic/Camera0.png')
#Canny 要求输入是单通道 8 位图，先转灰度
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
#90 是低阈值，150 是高阈值
can=cv2.Canny(gray,90,150)

while True:
    cv2.imshow('ori',img)
    cv2.imshow('new',can)
    key=cv2.waitKey(0)&0xff
    if key==ord('q'):
        break

cv2.destroyAllWindows()
```

注意：

- `cvtColor(img, cv2.COLOR_BGR2GRAY)`：把三通道 BGR 图转成单通道灰度图
- Canny 输出是 **0/255 的二值图**（255 是边缘，0 是背景），不是梯度图
- 调参经验：边缘多而杂 → 调大两个阈值；边缘太稀少 → 调小阈值

总结：三种边缘检测方法对比

方法	微分阶数	是否需要分方向	对噪声敏感度	边缘特点	适用场景
Sobel	一阶	需要（dx/dy）	较低（核自带平滑）	边缘较粗、有方向信息	快速检测、梯度计算
Laplacian	二阶	不需要（各向同性）	高	定位准但双响应、易受噪声干扰	配合高斯滤波（LoG）
Canny	一阶 + 多阶段	自动处理	低（先滤波）	单像素宽、连续、干净	最常用，通用边缘检测

思路

一阶 vs 二阶

- **一阶（Sobel）**：找”变化率最大”的地方，边缘处是**极大值**
- **二阶（Laplacian）**：找”变化率的拐点”，边缘处是**过零点**
- 二阶对细节更敏感，但也把噪声放大得更厉害，所以 Laplacian 常常和高斯滤波搭配使用

ddepth=-1 的坑　uint8 只能存 0~255，而梯度有正有负。ddepth=-1 直接截断负数，结果会”丢一半边缘”。**凡是求导操作，都建议用 cv2.CV_64F（或 CV_32F），最后再用 convertScaleAbs 转回 uint8 显示。**

边缘检测前的准备

- 先转灰度（Canny 强制要求）
- 先滤波去噪（高斯滤波最常用），否则噪声会被当成边缘
- 边缘检测是”前端操作”，后续常接轮廓查找 cv2.findContours、霍夫变换 cv2.HoughLines 等

什么时候用哪种?

```
需要梯度的方向和大小（如光流、角点检测的前置）  → Sobel
需要检测所有方向、细节敏感且噪声少        → Laplacian（或 LoG）
需要干净、连续、单像素的边缘轮廓            → Canny（首选）
需要自己调节边缘粗细和连续性                → Canny 调阈值
```

代码改进建议　把三份代码统一成”转灰度 + 取绝对值 + 加权合并”的写法，方便对比三种算子的效果：

```
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)           # 统一先转灰度
gx=cv2.Sobel(gray,cv2.CV_64F,1,0,ksize=-1)          # Scharr 更精确
gy=cv2.Sobel(gray,cv2.CV_64F,0,1,ksize=-1)
mag=cv2.addWeighted(cv2.convertScaleAbs(gx),0.5,cv2.convertScaleAbs(gy),0.5,0)
lap=cv2.convertScaleAbs(cv2.Laplacian(gray,cv2.CV_64F,ksize=3))
can=cv2.Canny(gray,90,150)
```

三张结果图放在一起对比，能明显看出：Sobel 边缘”粗”、Laplacian 边缘”碎”（噪声多）、Canny 边缘”细且连续”。